

Computer Programming and Problem Solving

Aaryan Khedekar

November 15, 2024

1 Introduction to C

1.1 History

C programming language was developed in 1972 by Dennis Ritchie at the bell laboratories in USA. It is extensively used as the most basic and fundamental programming language in today's world.

1.2 Constants

A constant is something that cannot be changed. There are two types of constants:

1. **Primary Constants:** int, float, char, etc.
2. **Secondary Constants:** struct, function, array, etc.

1.2.1 The range of some of the primary constants is given:

1. **int:** -32768 to 32767
2. **float:** $-34 * 10^{38}$ to $34 * 10^{38}$

1.3 Variables

A variable is something that can be changed. It can be a combination of characters, numbers, or underscores (-)

1.3.1 There are some rules for naming variables in C:

1. A variable name must only contain alphabets, digits, and underscore.
2. A variable name must start with an alphabet or an underscore only. It cannot start with a digit.
3. No white space is allowed within the variable name.
4. A variable name must not be any reserved word or keyword.

1.4 Keywords

A keyword is a predefined set of words performing a specific function. There are 32 predefined keywords in C.

Example: int, float, char, for, if, else, while, ...

1.5 Body of a C Program

At the top of a C program, all the header files (external libraries) are specified. The standard library "stdio.h" is included in nearly every file.

All the code that is to be executed is written in the "main()" function.

An example of a "Hello World" Program is given here:

```
#include <stdio.h>
main() {
    printf("Hello World!");
}
```

The "printf()" function is a part of the stdio.h library and hence that's the reason to include the library. At the end of all statements, a semicolon (;) is added to mark the end of the statement.

1.6 Program to calculate simple interest

```
#include <stdio.h>
main() {
    float pr, time, roi, si;
    pr = 3000;
    time = 3;
    roi = 4.2;
    si = pr * roi * time / 100;
    printf("The Simple Interest is"%f", si);
}
```

1.7 The scanf() function

This function is a part of the "stdio.h" library and is used to take input from the user. You need to specify which input you want from the user, by providing "%f" for float, "%d" for int, and "%c" for character. You also need to specify a variable pointer to which the user input value will be stored.

Syntax: scanf("%f", &variable);

1.8 Introduction to math.h library:

The math.h library is useful for working with mathematical functions, such as sin(), cos(), sqrt(), pow() and so on.

1.9 Types of Instruction

1. **Type Declaration:** Declaration of datatype to of the variable and initializing the value to the variable.
Example: `int a = 10;`
2. **Arithmetic Instructions:** These work on the mathematical problems based on addition, subtraction, multiplication, and division.
Example: `int a = b*2+c-1;` (if `b=5`, `c=2`, then `a=11`)
Note: No operator will be assumed. Example: $(a+b)^2 = a*a+b*b+2*a*b$
3. **Conditional Statements:** Only run if a certain condition is satisfied.
 - (a) **if statement:** IF the condition specified is satisfied, then statements inside the will be executed. Statements outside the if block will be executed regardlessly.
 - (b) **if-else statement:** IF the condition specified is satisfied, then statement inside the "if" block will be executed; otherwise the statements inside the "else" block will be executed.
 - (c) **Nested if-else statement:** Here the if/else statements are already inside of another if/else statement.
4. **Logical Operators:** These are used for two conditions inside of a if statements:
 - (a) **AND Operator (&&):** Returns true if both statements are true.
 - (b) **OR Operator (||):** Returns true if either statement is true.
 - (c) **NOT Opertor (!):** Returns true if the statement is false.

1.9.1 Some Example Programs:

1. Write a program for a user who is purchasing a product. If he purchases more than 1000 worth of product, give him a discount of 10%. DIisplay the total bill at the end.

```
#include <stdio.h>
main() {
    int qty;
    float rate, cost;
    printf("Enter the rate of the product: ");
    scanf("%f", &rate);
    printf("Enter the quantity of the product: ");
    scanf("%d", &qty);
    cost = rate*qty;
    if(cost > 1000) cost = 0.9*cost;
    printf("Your total bill is %f.", cost);
}
```

2. If salary of a person is less than 1500, his HRA is 10% and DA is also 10%. If it is more, or equal to 1500, then his HRA is 500 and DA is 98%. Display his gross salary. (GS = Sal + HRA + DA).

```
#include <stdio.h>
main() {
    float sal, hra, da, gs;
    printf("Enter your salary: ");
    scanf("%f", sal);
    if(sal < 1500) {
        hra = 0.1*sal;
        da = 0.1*sal;
    } else {
        hra = 500;
        da = 0.98*sal;
    }
    gs = sal + hra + da;
    printf("Your gross salary is %f.", gs);
}
```

3. A company gives loan based on some conditions to a driver. It gives loan if the driver is married **OR** the driver is unmarried, is a male, and is above 30 years **OR** the driver is unmarried, is a female, and is above 25 years. Otherwise the driver doesn't get a loan.

```
#include <stdio.h>
main() {
    int married, gender, age;
    printf("What is your marriage status?\n[0] Married\n[1] Unmarried\n");
    scanf("%d", &married);
    printf("What is your gender?\n[0] Male\n[1] Female\n");
    scanf("%d", &gender);
    printf("What is your age? ");
    scanf("%d", &age);
    if(married == 1) printf("You are eligible for the loan.");
    else {
        if(gender == 0 && age > 30) printf("You are eligible for the loan.");
        else if(gender == 1 && age > 25) printf("You are eligible for the loan.");
        else printf("You are ineligible for the loan.");
    }
}
```

1.9.2 Ternary Operator:

Ternary Operator (? :): A simplified version of the classic "if-else" statement, but in only one line.

Syntax: condition ? if true : if false;

1.10 Loops:

1. **while Loop:** while the condition is true, the statements inside the code block will be executed.

Syntax: while(condition) {...}

Question: Write a C Program to find the Simple Interest of 3 different inputs.

```
#include <stdio.h>
int main() {
    int pr, time, count=1;
    float roi, si;
    while(count<=3) {
        printf("Enter principal amount, duration, and rate
               of interest for iteration #%d: ", count);
        scanf("%d %d %f", &pr, &time, &roi);
        si = pr*time*roi;
        printf("The SI for iteration #%d is %f", count, si);
        count++;
    }
}
```

2. **for Loop:** Takes three arguments of variable initialization, condition and modification in one line instead of doing in multiple steps as in while loop.

Syntax: for(init counter; condition; modify counter) {...}

3. **do-while Loop:** It is similar to while loop, but the code inside the loop runs at least once before checking the condition.

Syntax: do {...} while (condition);

4. **return Keyword:** Simply exits the loop.

1.10.1 Some Example Problems:

1. Take input from the user and check if the input is a prime number or not.

```
#include <stdio.h>
int main() {
    int num, i, pr = 0;
    printf("Enter a number: ");
    scanf("%d", &num);
    for(i=2; i<num/2; i++) {
        if(num % i == 0) pr = 1;
    }
}
```

```

        printf("The number you entered is not prime.");
        return;
    }
    if(pr==0) printf("The number you entered is prime.")
}

```

2. Print the following using for loops:

```

1
1 2
1 2 3
1 2 3 4

```

```

#include <stdio.h>
int main() {
    for(int i=1; i<5; i++) {
        for(int j=1; j<=i; j++) {
            printf("%d ", j);
        }
        printf("\n");
    }
}

```

1.11 switch-case Statement:

1. It is a more efficient way to write a standard "if-else" statement, as instead of going through all the if conditions, here it directly jumps to the case.

2.

```

switch(variable) {
    case constant:
        // if variable = constant, then the code executes
        break; // to come out of the switch
    case another_constant:
        // code here
        break;
    default:
        // if none of the cases match, this code executes
        break;
}

```

3. Only integers and characters can be the constants in a switch-case statement. floats can't be used.

4. **Question:** Using switch-case statement, create a simple calculator taking two integer inputs and one operator input from the user.

```
#include <stdio.h>
int main() {
    int a, b, op;
    printf("Enter two integers: ");
    scanf("%d %d", &a, &b);
    printf("[0] Addition\n[1] Subtraction\n[2]
    Multiplication\n[3] Division\nEnter the operation
    to perform: ");
    scanf("%d", &op);
    switch(op) {
        case 0:
            printf("%d + %d = %d", a, b, a+b);
            break;
        case 1:
            printf("%d - %d = %d", a, b, a-b);
            break;
        case 2:
            printf("%d * %d = %d", a, b, a*b);
            break;
        case 3:
            if(b != 0) {
                printf("%d / %d = %f", a, b, a/b);
            } else printf("Cannot divide by 0");
            break;
        default:
            printf("Invalid input provided.");
            break;
    }
}
```

1.12 Functions:

1. Functions are useful to break the control of the program and execute a series of commands outside the main control.
2. To declare a function, you need to initiate it with a datatype (void, int, char, float) before the main function.
3. If the function requires some arguments, then the datatype of those args must be provided at the time of declaration.
4. For defining the function use, you can do it within the main function or outside it.
5. To call the function, "function_name();" syntax is used.

6. **Question:** Using functions, make a simple calculator that runs until the user stops.

```
#include <stdio.h>
int calc(int, int, int, int);
int main() {
    int a, b, op, run = 0;
    do {
        printf("Enter the first number: ");
        scanf("%d", &a);
        printf("Enter the second number: ");
        scanf("%d", &b);
        printf("[0] Addition\n[1] Subtraction\n[2] Multiplication\n[3] Division\n Enter the operation to be performed: ");
        scanf("%d", &op);
        run = calc(a, b, op, run);
    } while(run == 1);
}

int calc(int a, int b, int op, int run) {
    int _run;
    switch(op) {
        case 0:
            printf("%d + %d = %d", a, b, a+b); break;
        case 1:
            printf("%d - %d = %d", a, b, a-b); break;
        case 2:
            printf("%d * %d = %d", a, b, a*b); break;
        case 3:
            printf("%d / %d = %f", a, b, a/b); break;
    }
    printf("\n[0] No\n[1] Yes\n Run Again??: ");
    scanf("%d", &_run);
    run = _run;
    return run;
}
```

1.13 Pointers:

1. A pointer is a variable that stores the address of another variable.
2. "&var_name" denotes the address of a variable.
Example: printf("%u", &var_name); displays _address of the variable
Example: printf("%d", var_name); displays _value of the variable
3. A pointer is defined by a "*" before the variable name.

4. Example:

```
#include <stdio.h>
int main() {
    int i = 3, *j; // declares a variable and a pointer.
    j = &i; // set pointer j to variable i.
    printf("%u", j); // prints the address of "i".
    printf("%d", *j); // prints the value of "i".
    printf("%u", &j); // prints the address of "j".
}
```

5. **Call by Value:** A function is called with some arguments, but after the end of the function, the value of the arguments doesn't change.

```
#include <stdio.h>
int main() {
    int a = 2, b = 3;
    swap(a, b); // Call by Value.
}

void swap(int x, int y) {
    int temp = x;
    x = y;
    y = temp;
    printf("%d %d", x, y);
    // We need to print values inside the functions for
    // proper output. If we print values inside the main()
    // function, the values would print unswapped.
}
```

6. **Call by Reference:** A function is called with some arguments, but after the end of the function, the values of the arguments changes.

```
#include <stdio.h>
void swap(int *, int *);
int main() {
    int a = 2, b = 3;
    swap(&a, &b); // Call by Reference.
    printf("%d %d", a, b);
    // Print function can be called after swap function
    // inside main() as the values of a and b are directly
    // edited from third address using pointers.
}

void swap(int *x, int *y) {
    // Pointers are used to directly edit values of a and b.
    int temp = *x;
    *x = *y;
    *y = temp;
}
```

7. **Question:** Write a C program to find the area and circumference of a circle given its radius using function call by reference.

```
#include <stdio.h>
void calc(int, float *, float *);
int main() {
    int radius;
    float *area, *circum;
    printf("Enter the value of radius: ");
    scanf("%d", &radius);
    calc(radius, &area, &circum);
    printf("For a circle of radius %d, area is %.2f and
           circumference is %.2f", radius, *area, *circum);
}
void calc(int r, float *a, float *c) {
    *a = 3.14 * r * r;
    *c = 2 * 3.14 * r;
}
```

1.14 Ranges of C DataTypes:

1. **int:** -2^{31} to $2^{31} - 1$ (-2,147,483,648 to 2,147,483,647)
2. **short:** -2^{15} to $2^{15} - 1$ (-32,768 to 32,767)
3. **long:** -2^{63} to $2^{63} - 1$ (-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807)
4. **unsigned int:** 0 to $2^{32} - 1$ (0 to 4,294,967,295)
5. **unsigned short:** 0 to $2^{16} - 1$ (0 to 65,535)
6. **unsigned long:** 0 to $2^{64} - 1$ (0 to 18,446,744,073,709,551,615)
7. **char:** -2^7 to $2^7 - 1$ (-128 to 127)
8. **unsigned char:** 0 to $2^8 - 1$ (0 to 255)
9. **float:** -3.4×10^{38} to 3.4×10^{38}
10. **double:** -1.7×10^{308} to 1.7×10^{308}

1.15 Storage Classes:

1. **Automatic:** This is the default storage class. It stores the value in the main memory. The default value is garbage. Scope of the variable is local.
Syntax: auto int i;
2. **Register:** It stores the value in the CPU. The default value is garbage. Scope of the variable is local.
Syntax: register int i;

3. **Static:** It stores the value in the main memory. The default value is zero. Once a variable is initialized, it can't be initialize again. Scope of the variable is local.
Syntax: static int i;
4. **External:** It stores the value in the main memory. The default value is zero. Scope of the variable is global.
Syntax: extern int i;

1.16 Macros:

1. This is similar to functions but instead of passing the control to the function, the macro code is directly replaced in the code during preprocessing.
2. **Syntax:** #define MACRO_NAME snippet
3. The macro will be declared as a "keyword" and not a "variable".
4. Macros will not consume memory space.
5. The value of macro will not change.
6. **Argumented Macros:** Arguments can be passed through the macros similar to functions.
7. **Question:** Using macros, make a C Program to calculate the Area of a Circle provided the radius.

```
#include <stdio.h>
#define PI 3.142
#define AREA(r) (PI*r*r)
int main() {
    int radius;
    printf("Enter the value of radius: ");
    scanf("%d", &radius);
    printf("The area of the circle is: %.3f", AREA(radius));
}
```

1.17 Arrays:

1. Arrays are a linear collection of items of same datatype under a single variable name.
2. **Syntax:** int array[size] = {items...};
3. An array starts with the index of "zero" and not one. Hence, the first item will be array[0].
4. To access the value of an element, we specify the index of the element in the array.

5. **Example:**

```
int array[5] = {1, 2, 3, 4, 5};
printf("%d", array[3]); // this will print out "4".
```

6. **Question:** Given the marks (out of 100) of 5 subjects from the user, calculate the total percentage obtained by the user.

```
#include <stdio.h>
int main() {
    int marks[5], sum=0;
    printf("Enter marks of 5 subjects: ");
    for(int i=0; i<5; i++) scanf(" %d", &marks[i]);
    for(int i=0; i<5; i++) sum+=marks[i];
    printf("The percentage of marks obtained by the user is
           %f.", sum/5);
}
```

7. **Passing values of array elements to function:**

```
#include <stdio.h>
int main() {
    int a[5] = {1,2,3,4,5};
    for(int i=0; i<5; i++) value_to_fn(a[i]);
}

void value_to_fn(int b) { printf("%d ", b); }
```

Here only the values of the elements are passed one by one to the function.

8. **Passing entire array to function:**

```
#include <stdio.h>
int main() {
    int a[5] = {1,2,3,4,5};
    array_to_fn(&a[0], 5);
}

void array_to_fn(int *b, int size) {
    for(int i=0; i<size; i++) {
        printf("%d ", *b);
        b++;
    }
}
```

Here the first address of the array and the size of the array is passed onto the function, and the address of the elements increases in the function itself using the size provided.

1.18 2-D Arrays:

1. **Question:** Declare a matrix "cos matrix" if the first element is bigger than the second, second one is smaller than third, third is bigger than fourth and so on. Check if a given 3x3 matrix is cos matrix or not.

```
#include <stdio.h>

int main() {
    int a[4][4], is = 1, n;
    printf("Enter 3x3 matrix:\n");
    for(int i = 1; i < 4; i++) {
        for(int j = 1; j < 4; j++) scanf("%d", &a[i][j]);
    }
    n = a[1][1];
    for(int i = 1; i < 4; i++) {
        for(int j = 1; j < 4; j++) {
            if((i+j)%2 == 0 && n > a[i][j]) is = 0;
            if((i+j)%2 != 0 && n < a[i][j]) is = 0;
            n = a[i][j];
        }
    }
    if(is == 0) printf("\nThe given matrix is not a cos
matrix.");
    else printf("\nThe given matrix is a cos matrix.");
}
```

2. **Question:** Consider cos matrix as declared above. Declare a "cos path" which is the sum of all element until the "cos matrix" pattern is followed in a given matrix. Find the cos path length of a given 3x3 matrix.

```
#include <stdio.h>

int main() {
    int a[4][4], is = 1, n, pl = 0;
    printf("Enter 3x3 matrix:\n");
    for(int i = 1; i < 4; i++) {
        for(int j = 1; j < 4; j++) scanf("%d", &a[i][j]);
    }
    n = a[1][1];
    for(int i = 1; i < 4; i++) {
        for(int j = 1; j < 4; j++) {
            if((i+j)%2 == 0 && n > a[i][j]) is = 0;
            if((i+j)%2 != 0 && n < a[i][j]) is = 0;
            n = a[i][j];
            if(is == 1) pl += n;
        }
    }
    if(is == 0) printf("\nThe given matrix is not a cos
```

```

        matrix and its path length is %d", pl);
    else printf("\nThe given matrix is a cos matrix and its
        path length is %d", pl);
}

```

1.19 Strings:

1. A string is an "array of characters".
2. **Syntax:** `char a[size] = "string";`
3. A string is ended by a null character (`'\0'`) which consumes one byte in the memory. Hence, size of string = (number of characters + 1) bytes.
4. Read/Print an array using `scanf()` and `printf()`:

```

int main() {
    char a[20];
    for(int i = 0; a[i] != '\0'; i++) scanf("%c", &a[i]);
    for(int i = 0; a[i] != '\0'; i++) printf("%c", a[i]);
}

```

5. Read/Print an array using pointers:

```

int main() {
    char a[] = "Hello", *p;
    p = a;
    for(int i = 0; *(p+i) != '\0'; i++) printf("%c",
        *(p+i));
}

```

6. Read/Print an array using "%s" literal:

```

int main() {
    char a[20];
    scanf("%s", a);
    printf("%s", a);
}

```

7. Using `scanf()` and `printf()` you can't work with multi-word strings as they terminate at spaces. To fix this, we use the `gets()` and `puts()` functions.

```

int main() {
    char a[20];
    gets(a);
    puts(a);
}

```

1.19.1 Some Inbuilt Functions of string.h

1. **strlen():** Gives the length of a given string.
Syntax: int strlen(char[]);
2. **strcpy():** Copies the contents of one string to another.
Syntax: void strcpy(char[] target, char[] source);
3. **strcat():** Concatenates two strings.
Syntax: void strcat(char[] target, char[] source);
4. **strcmp():** Compares two strings and give the difference between the ASCII values of the first mismatched character.
Syntax: int strcmp(char[] str1, char[] str2);
5. Some more functions include: strlwr(),strupr(), strcmpi(), strdup(), strchr(), strrchr(), strstr(), strset(), strrev(), etc.

1.19.2 2-D Array of Characters (Array of Strings):

1. **Syntax:** char a[optional size][mandatory size];
2. *a[i] is a 2-D array and is an array of pointers.
3. (*a)[i] is a 1-D array and is an array of values.
4. **Question:** Write a program to find if a string is in a given list of strings.

```
#include <stdio.h>
#include <string.h>

int main() {
    char list[][10] = {"Hello", "World", "Test", "Array"};
    char a[10];
    int is = 0;
    gets(a);
    for(int i = 0; i < 4; i++) {
        if(strcmp(a, list[i][0]) == 0) is = 1;
    }
    if(is == 0) printf("Name Not Found.");
    else printf("Name Found.");
}
```
